

TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



HỌC PHẦN: NGÔN NGỮ LẬP TRÌNH C#
BÁO CÁO ĐỒ ÁN

ĐỀ TÀI: MEDIA STREAMING WEB APPLICATION

Giảng viên hướng dẫn: Thầy Từ Lăng Phiêu

Nhóm sinh viên thực hiện

Chế Văn Nam - 3124410217

Nguyễn Thành Nhân - 3124410249

Vũ Lê Phước Sang - 3124410303

TP Hồ Chí Minh, Tháng 6 Năm 2026

NHẬN XÉT CỦA GIẢNG VIÊN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

...

LỜI CẢM ƠN

Chúng em xin chân thành cảm ơn đến thầy **Từ Lăng Phiêu** – giảng viên bộ môn “Ngôn ngữ lập trình C#” thuộc Khoa Công Nghệ Thông Tin, trường Đại học Sài Gòn, đã trang bị cho chúng em những kiến thức, kỹ năng cơ bản cần có để có thể hoàn thành đồ án này.

Tuy nhiên, trong quá trình hoàn thiện đồ án, với vốn kiến thức cũng như kinh nghiệm còn rất khiêm tốn và là bước đầu làm quen với công việc nghiên cứu mang tính thực nghiệm, chúng em vẫn còn nhiều thiếu sót, hạn chế trong việc tìm hiểu và xây dựng đồ án về này. Rất mong được sự quan tâm, góp ý của thầy để đồ án của chúng em được đầy đủ và hoàn chỉnh hơn.

Xin kính chúc thầy **Từ Lăng Phiêu** dồi dào sức khỏe và hạnh phúc để tiếp tục thực hiện sứ mệnh cao đẹp của mình là truyền đạt kiến thức cho thế hệ mai sau.

Xin chân thành cảm ơn.

Tp.HCM ngày 26 tháng 06 năm 2026

Sinh viên thực hiện

Chế Văn Nam

Nguyễn Thành Nhân

Vũ Lê Phước Sang

Mục lục

LỜI CẢM ƠN	3
1. Tổng quan và kiến trúc hệ thống	5
1.1 Tổng quan về dự án TuneVault	5
1.2 Kiến trúc Clean Architecture 4 tầng	5
1.3 Cơ chế CQRS kết hợp MediaR Pipeline Behavior	5
2. Lý do chọn Dapper ORM thay vì Entity Framework Core	6
2.1 Ưu điểm khi lựa chọn Dapper ORM	6
2.2 Nhược điểm của Entity Framework (EF) Core trong đồ án	6
3. Mô tả 10 chức năng chính của hệ thống	7
3.1 Chức năng xác thực:	7
3.2 Chức năng Profile:	7
3.3 Chức năng Media Library:	7
3.4 Chức năng Audio Player:	8
3.5 Chức năng Video Player:	8
3.6 Chức năng PlayList:	8
3.7 Chức năng tìm kiếm & khám phá:	8
3.8 Chức năng chia sẻ Media:	8
3.9 Chức năng thông báo:	8
3.10 Chức năng Tương tác & lịch sử:	9
4. Khó khăn và hướng giải quyết	9
4.1 Khó khăn về cấu hình Real-Time(SignalR) trên môi trường Cloud:	9
4.2 Khó khăn trong việc xử lý logic ràng buộc phức tạp	9
4.3. Khó khăn về cấu hình CORS khi kết nối Frontend và Backend	10
4.4 Khó khăn trong quản lý mã nguồn nhóm (Merge Conflict)	10
TỔNG KẾT	11
TÀI LIỆU THAM KHẢO	12

1. Tổng quan và kiến trúc hệ thống

1.1 Tổng quan về dự án TuneVault

Hệ thống TuneVault là một ứng dụng phát nhạc trực tuyến (Streaming Music) hiện đại, hướng tới trải nghiệm người dùng tối giản, tốc độ tải trang nhanh và bảo mật cao. Hệ thống được chia tách rạch ròi giữa tầng giao diện hiển thị (Frontend phát triển bằng React, TypeScript) và tầng logic xử lý trung tâm (Backend được xây dựng trên nền tảng .Net Core

1.2 Kiến trúc Clean Architecture 4 tầng

- **Domain (Lõi hệ thống):** Chứa các thực thể doanh nghiệp thuần túy (Entities) như User, Song, Playlist. Lớp này tinh khiết 100%, không phụ thuộc vào bất kỳ thư viện hay cơ sở dữ liệu nào bên ngoài.
- **Application (Nghệp vụ):** Chứa các khuôn mẫu dữ liệu (DTOs), các Interface và các bộ xử lý chính (**Handlers**). Lớp này sử dụng thư viện MediatR để điều phối toàn bộ luồng đi của ứng dụng.
- **Infrastructure (Hạ tầng):** Nơi chứa code hiện thực hóa các công nghệ bên ngoài như kết nối SQL Server thông qua Dapper, gọi API Clouinary, hoặc chạy thuật toán mã hóa mã vạch, băm mật khẩu.
- **Presentation (API / Controllers):** Cổng đón tiếp yêu cầu (Request) từ giao diện React gửi sang, bóc tách thông tin bảo mật sơ bộ và ném xuống cho tầng Application xử lý.

1.3 Cơ chế CQRS kết hợp MediaR Pipeline Behavior

Hệ thống phân tách rạch ròi thành hai luồng dữ liệu độc lập: **Command** (Mệnh lệnh làm thay đổi dữ liệu như Thêm/Sửa/Xóa) và **Query** (Yêu cầu đọc dữ liệu lên màn hình). Trước khi một mệnh lệnh chạm được vào các Handler xử lý dưới Database, nó phải đi qua hai trạm gác tự động:

- **ValidationBehavior:** Tự động gọi các bộ quy tắc FluentValidation để soi lỗi cú pháp (Email sai định dạng, tên trống...). Nếu sai, nó bẻ gãy luồng chạy và dội ngược lỗi về Controller ngay lập tức để tiết kiệm tài nguyên hệ thống.
- **AuthorizationBehavior:** Trạm gác xét duyệt quyền hạn. Nó bóc tách JWT Token để kiểm tra xem người dùng có phải là chủ sở hữu hợp

pháp của tài nguyên đó hay không (Ví dụ: Chỉ chủ của Playlist mới được xóa bài hát trong đó).

2. Lý do chọn Dapper ORM thay vì Entity Framework Core

Trong quá trình phát triển hệ thống TuneVault, nhóm nghiên cứu đã tiến hành phân tích và thử nghiệm hai giải pháp ánh xạ cơ sở dữ liệu phổ biến là **Entity Framework (EF) Core** (Heavyweight ORM) và **Dapper ORM** (Micro-ORM). Kết quả cho thấy, việc lựa chọn Dapper ORM mang lại những giá trị vượt trội về cả tư duy thiết kế lẫn hiệu năng thực tế cho đồ án.

2.1 Ưu điểm khi lựa chọn Dapper ORM

- **Làm chủ hoàn toàn câu lệnh SQL và Thiết kế hệ thống:** Việc không sử dụng cơ chế tự động sinh bảng (Migrations) của EF Core ép buộc người phát triển phải tự tay thiết kế Schema, tự viết các ràng buộc toàn vẹn (FOREIGN KEY, UNIQUE, CHECK) trực tiếp trên SQL Server. Điều này giúp người lập trình hiểu sâu sắc cấu trúc lưu trữ vật lý và nâng cao tư duy tối ưu hóa cơ sở dữ liệu ngay từ bước thiết kế ban đầu.
- **Hiệu năng đạt mức tối đa (Performance Optimization):** Dapper là một Micro-ORM siêu nhẹ, hoạt động theo cơ chế ánh xạ trực tiếp (Direct Mapping). Khác với các ORM hạng nặng, Dapper loại bỏ hoàn toàn cơ chế theo dõi trạng thái thực thể (**Change Tracker** - cơ chế liên tục giám sát dữ liệu trong RAM để xem có sự thay đổi nào không). Khi nhận lệnh, Dapper chỉ thực thi câu lệnh SQL thuần túy, đổ thẳng dữ liệu vào class C# rồi giải phóng bộ nhớ RAM ngay lập tức. Nhờ vậy, tốc độ thực thi truy vấn đạt mức tối đa (xấp xỉ ADO.NET thuần túy), RAM máy chủ luôn nhẹ thoáng và tiết kiệm tài nguyên CPU tối đa khi xử lý các luồng stream nhạc hoặc dữ liệu lớn.
- **Kiểm soát 100% mã nguồn truy vấn (Absolute Query Control):** Với Dapper, người lập trình chủ động viết và kiểm soát từng dòng lệnh SQL gửi xuống cơ sở dữ liệu. Ta có quyền quyết định việc kết hợp bảng nào trước (JOIN), chọn lọc chính xác những cột dữ liệu thực sự cần thiết để truyền tải thay vì lấy thừa thãi, và tận dụng triệt để các chỉ mục vật lý (Index) đã thiết lập dưới Database. Dữ liệu trả về sẽ chạy chính xác 100% theo ý đồ của người viết, không bao giờ phát sinh các câu lệnh SQL rác hoặc câu lệnh ngầm chạy ẩn sau lưng gây bất ngờ hay làm chậm hệ thống.

2.2 Nhược điểm của Entity Framework (EF) Core trong đồ án

- **Làm suy giảm hiệu năng do bộ máy quản lý cổng kênh:** EF Core duy trì cơ chế *Change Tracker* để giám sát trạng thái của các thực thể trong RAM. Quá trình này tiêu tốn rất nhiều tài nguyên bộ nhớ đệm của máy chủ. Ngoài ra, việc dịch từ ngôn ngữ LINQ C# sang câu lệnh SQL thuần cũng làm tiêu hao đáng kể tài nguyên CPU của Server, khiến tốc độ phản hồi API bị chậm hơn so với Micro-ORM.
- **Đánh mất quyền kiểm soát câu lệnh thực tế:** Do câu lệnh SQL được EF Core sinh ra tự động, người lập trình hoàn toàn bị động và không biết chính xác cơ sở dữ liệu đang thực thi cái gì. Đối với các mối quan hệ phức tạp lồng nhau (như tính toán Followers/Following động), EF Core thường tự động sinh ra các câu lệnh gộp bảng (JOIN) cực kỳ cổng kênh, kém tối ưu, hoặc tệ hơn là rơi vào lỗi **N+1 Query** (chọc xuống DB lặp đi lặp lại nhiều lần vô tội vạ) làm nghẽn mạch đường truyền và kéo sập hiệu năng của toàn bộ hệ thống.
- **Tạo tâm lý phụ thuộc và mờ nhạt tư duy Database:** Cơ chế tự động tạo bảng (Migrations) và tự viết SQL của EF Core vô tình tạo ra tâm lý ỷ lại vào công cụ. Lập trình viên sẽ mất đi thói quen tương tác trực tiếp với hệ quản trị CSDL, dẫn đến việc không hiểu rõ bản chất cấu trúc cơ sở dữ liệu quan hệ, gây khó khăn lớn khi hệ thống cần mở rộng hoặc tối ưu nâng cao.

3. Mô tả 10 chức năng chính của hệ thống

3.1 Chức năng xác thực:

Quản lý luồng đăng ký và đăng nhập. Sử dụng thuật toán BCrypt để đổi mật khẩu(Hashing) trước khi lưu vào cơ sở dữ liệu.

Cấp phát JWT Token vô trạng thái để xác thực người dùng trong các phiên làm việc tiếp theo.

3.2 Chức năng Profile:

Cho phép người dùng xem và chỉnh sửa thông tin cá nhân(Tên, Tiểu sử).

Tích hợp dịch vụ Cloudinary để hỗ trợ tính năng cập nhật ảnh đại diện.

3.3 Chức năng Media Library:

Hỗ trợ người dùng tải lên (Upload) các tệp âm thanh(mp3) và video (mp4) đi kèm metadata (Tựa đề, Thể loại, nghệ sĩ).

Backend sử dụng luồng MultipartFormDataStream để xử lý các tệp dung lượng lớn mà không gây quá tải bộ nhớ.

3.4 Chức năng Audio Player:

Cung cấp thanh điều khiển toàn cục dưới cùng màn hình (Play, Pause, seek, volume, Queue, Friends).

Hệ thống tự động kích hoạt RecordPlayHistoryCommand ngầm khi bài hát được phát để ghi nhận lịch sử.

3.5 Chức năng Video Player:

Phát video trong giao diện Modal Popup nổi bật.

API Backend hỗ trợ Range Request cho phép trình duyệt chỉ tải trước một đoạn ngắn của video, giúp xem mượt mà không cần đợi tải toàn bộ tệp.

3.6 Chức năng PlayList:

Cho phép thực hiện các thao tác CRUD lên Playlist.

Cấu hình AuthBehavior trong pipeline để chặn đứng các hành vi thêm/xóa bài hát trái phép nếu người thao tác không phải là chủ sở hữu (Owner) của Playlist đó.

3.7 Chức năng tìm kiếm & khám phá:

Tìm kiếm chéo dựa trên nhiều thực thể (Song, Artist, Playlist).

Dữ liệu trả về được áp dụng kỹ thuật phân trang (pagination) để tối ưu hóa thời gian phản hồi của API.

3.8 Chức năng chia sẻ Media:

Người dùng có thể gửi bài hát hoặc danh sách phát trực cho tài khoản khác và có thể nghe nhạc mà người khác gửi qua cho mình dựa vào hệ thống Inbox.

Áp dụng Validator để chống lại các hành vi logic lỗi (Ví dụ: Chặn người dùng tự chia sẻ cho chính mình).

3.9 Chức năng thông báo:

Sử dụng công nghệ SignalR kết hợp với Background Queue.

Bất cứ khi nào có hành động chia sẻ (Share) hoặc Theo dõi (Follow), hệ thống lưu thông báo vào CSDL và lập tức Push tín hiệu về Frontend để hiển thị Popup mà không cần tải lại trang.

3.10 Chức năng Tương tác & lịch sử:

Cung cấp các thao tác tương tác mạng xã hội: Thích (Like/ Favorite) bài hát và theo dõi nghệ sĩ.

Giao diện (UI) sử dụng kỹ thuật Optimistic UI Update (Đổi màu trái tim ngay lập tức trên Frontend) để tăng cường trải nghiệm mượt mà, sau đó mới đồng bộ dữ liệu ngầm xuống Backend.

4. Khó khăn và hướng giải quyết

Trong quá trình phát triển và triển khai hệ thống, nhóm đã gặp phải một số thách thức mang tính kỹ thuật sâu và đã đưa ra các giải pháp xử lý cụ thể:

4.1 Khó khăn về cấu hình Real-Time(SignalR) trên môi trường Cloud:

Vấn đề: Khi chạy dưới Cloud, SignalR hoạt động trơn tru qua tính năng Proxy của Vite. Tuy nhiên, khi Deploy Frontend lên Vercel và Backend lên Somee Cloud, các kết nối WebSockets liên tục bị từ chối do Vercel không hỗ trợ chuyển tiếp WebSockets, kèm theo lỗi bảo mật Mixed Content (Frontend https bị chặn khi gọi Backend http).

Hướng xử lý: Nhóm đã can thiệp vào tầng cấu hình Axios và SignalR Connection Builder trên Frontend. Thay đổi đường dẫn tương đối thành URL tuyệt đối trỏ thẳng về Somee Cloud. Đồng thời, bổ sung thẻ <meta http-equiv="Content-Security-Policy" content="upgrade-insecure-requests"> vào tệp index.html để ép trình duyệt nâng cấp luồng dữ liệu mạng vượt qua rào cản CORS và bảo mật.

4.2 Khó khăn trong việc xử lý logic ràng buộc phức tạp

Vấn đề: ở các chức năng như chia sẻ hay thêm bài vào playlist, có quá nhiều điều kiện cần kiểm tra (Token có hợp lệ hay không? ID bài hát có tồn tại hay không? Người nhận có là chính mình hay không? Có phải là chủ sở hữu Playlist không?). Nếu nhồi nhét tất cả các lệnh if/else này vào controller hoặc service thì code sẽ trở nên khổng lồ và khó bảo trì

Hướng xử lý: Vận dụng triệt để sức mạnh của thư viện MediaR trong Clean Architecture. Nhóm tự xây các IPipelineBehavior(như AuthBehavior, ValidationBehavior). Dữ liệu request từ controller trước khi đến được tầng xử lý lỗi(Handler) bắt buộc phải đi qua “lớp vỏ” pipeline này. Bất cứ lỗi logic nào cũng sẽ bị pipeline chặn đứng và văng ra HTTP 400/403 giúp cho code bên trong Handler trở nên tinh gọn, chỉ tập trung vào nghiệp vụ CSDL

4.3. Khó khăn về cấu hình CORS khi kết nối Frontend và Backend

Vấn đề: Trong giai đoạn phát triển, Frontend React chạy trên cổng 5174 trong khi Backend API chạy trên cổng 5000. Trình duyệt tự động chặn toàn bộ các Request gọi chéo nguồn gốc theo chính sách bảo mật mặc định khiến mọi lệnh gọi API đều bị từ chối với lỗi CORS policy blocked trên console trình duyệt.

Hướng xử lý: Nhóm cấu hình CORS Middleware trong tầng API(Program.cs) để khai báo chính xác danh sách các Origin được phép, môi trường Local cho phép <http://localhost:5174>, môi trường Production thêm domain Vercel. Phía Frontend sử dụng tính năng Proxy của Vite để chuyển hướng toàn bộ Request có tiền tố /api sang Backend, giúp trình duyệt không nhận ra đây là Cross-Origin Request và bỏ qua kiểm tra CORS trong môi trường phát triển

4.4 Khó khăn trong quản lý mã nguồn nhóm (Merge Conflict)

Vấn đề: khi các thành viên cùng code chung giao diện hoặc chỉnh sửa cấu trúc CSDL, hiện tượng merge conflict xuất hiện liên tục

Hướng xử lý: Áp dụng git Branching nghiêm ngặt. Đặc biệt, nhóm đã tự nghiên cứu và tích hợp thành công CI/CD Pipeline bằng GitHub Actions. Bất kỳ đoạn code nào khi đẩy lên nhánh main đều phải trải qua vòng kiểm tra tự động docnet build. Lỗi biên dịch sẽ bị phát hiện ngay lập tức trước khi được phép gộp vào main

TỔNG KẾT

Hệ thống nghe nhạc trực tuyến TuneVault là một nền tảng đa phương tiện hiện đại, được xây dựng tối ưu theo mô hình kiến trúc phân tầng **Clean Architecture** kết hợp mẫu thiết kế CQRS và Micro-ORM Dapper, nhằm đảm bảo tính tách biệt rõ ràng giữa các tầng nghiệp vụ, tối ưu hóa hiệu năng truy vấn và dễ dàng mở rộng trong tương lai. Hệ thống đã hoàn thiện và cung cấp đầy đủ 10 chức năng cốt lõi bao gồm xác thực người dùng, phát nhạc trực tuyến, quản lý danh sách phát, tìm kiếm nội dung, hệ thống theo dõi và thông báo thời gian thực. Việc ứng dụng công nghệ SignalR (WebSockets) giúp người dùng nhận thông báo tương tác tức thì mà không cần tải lại trang, trong khi kỹ thuật phân đoạn dữ liệu HTTP 206 Partial Content đảm bảo việc truyền tải luồng âm thanh mượt mà, ổn định ngay cả với các tệp tin dung lượng lớn. Toàn bộ hệ thống đã được triển khai thành công lên môi trường trực tuyến với Backend trên nền tảng Somee Cloud và Frontend trên Vercel, đồng thời tích hợp quy trình CI/CD tự động qua GitHub Actions, minh chứng cho khả năng kiểm soát chất lượng mã nguồn và phối hợp hiệu quả trong suốt quá trình phát triển nhóm.

HƯỚNG PHÁT TRIỂN

- **Nâng cấp cơ chế xác thực:** Bổ sung cơ chế làm mới phiên đăng nhập bằng Refresh Token (JWT ngắn hạn kết hợp Refresh Token dài hạn lưu trữ an toàn dưới cơ sở dữ liệu) thay thế cho cơ chế JWT đơn thuần hiện tại, giúp tăng cường tính bảo mật và cải thiện trải nghiệm người dùng, tránh tình trạng bị ngắt phiên làm việc đột ngột.
- **Tối ưu hóa hiệu năng bằng bộ đệm (Cache):** Tích hợp hệ thống Redis Cache để lưu trữ tạm thời kết quả của các truy vấn có tần suất lặp lại cao như danh sách bài hát nổi bật, bảng xếp hạng hoặc kết quả tìm kiếm phổ biến. Giải pháp này giúp giảm tải áp lực truy vấn trực tiếp lên SQL Server vật lý và cải thiện đáng kể tốc độ phản hồi API.
- **Mở rộng tính năng trí tuệ nhân tạo (AI Recommendation):** Phát triển thêm tính năng gợi ý bài hát cá nhân hóa dựa trên lịch sử nghe nhạc của từng cá nhân. Sử dụng thuật toán lọc cộng tác hoặc tích hợp các mô hình học máy để phân tích sâu gu âm nhạc, từ đó đề xuất nội dung phù hợp theo thời gian thực.
- **Phát triển ứng dụng di động đa nền tảng:** Mở rộng hệ sinh thái sang ứng dụng di động (chạy trên cả hai hệ điều hành iOS và Android) bằng framework React Native. Hướng đi này cho phép tái sử dụng lại toàn bộ mã nguồn Logic của hệ thống và hệ thống API Backend hiện có, giúp người dùng tiếp cận thư viện nhạc mọi lúc mọi nơi trên thiết bị cầm tay.

TÀI LIỆU THAM KHẢO

- [1] ASP.NET Core: Nền tảng cốt lõi xây dựng Web API tại <https://learn.microsoft.com/aspnet/core>
- [2] Entity Framework Core: Công cụ ORM cơ sở tại <https://learn.microsoft.com/ef/core>
- [3] Dapper: Micro-ORM để tối ưu tốc độ truy vấn SQL tại <https://github.com/DapperLib/Dapper>
- [4] MediatR (Pipeline Pattern): Quản lý CQRS tại <https://github.com/jbogard/MediatR>
- [5] FluentValidation: Thư viện Validate dữ liệu tại <https://docs.fluentvalidation.net>
- [6] Clean Architecture (Jason Taylor template): Khuôn mẫu kiến trúc dự án Backend tại <https://github.com/jasontaylordev/CleanArchitecture>
- [7] SignalR: Triển khai WebSocket để đẩy thông báo thời gian thực tại <https://learn.microsoft.com/aspnet/core/signalr>
- [8] Anthropic API (Claude) / Google Gemini API: Tích hợp tính năng AI Chatbot thông minh tại <https://docs.anthropic.com/en/api/getting-started>
- [9] React: Thư viện giao diện Frontend tại <https://react.dev>
- [10] Vite: Công cụ Build Tool cực nhanh cho SPA tại <https://vitejs.dev>
- [11] Rubric B1 ("Clean Architecture & Cấu trúc solution"): Áp dụng chi tiết các tiêu chí chấm điểm cho mục B1 để xác định 4 project cốt lõi, Dependency Rule, kỹ thuật Dependency Injection (DI) và việc bóc tách triệt để logic nghiệp vụ.